arXiv:2605.10332v2 [cs.AI] 11 Jul 2026

EmbodiSkill: Skill-Aware Reflection for Self-Evolving Embodied Agents

Ruofei Ju[1*†] **Xinrui Wang**[2*†] **Xin Ding**[3‡] **Yifan Yang**[4] **Hao Wu**[1] **Shiqi Jiang**[4] **Qianxi Zhang**[4]
Hao Wen[5] **Xiangyu Li**[5] **Weijun Wang**[5] **Kun Li**[5] **Yunxin Liu**[5] **Haipeng Dai**[1] **Wei Wang**[1] **Ting Cao**[5‡]

[1] Nanjing University [2] Huazhong University of Science and Technology [3] University of Science and Technology of China
[4] Microsoft Research [5] Institute for AI Industry Research (AIR), Tsinghua University

Embodied agents can benefit from skills that guide object search, action execution, and state changes across diverse environments. Since embodied environments vary across layouts, object states, and other execution factors, these skills must self-evolve from trajectories generated during task execution. However, existing skill self-evolution methods are mainly developed in digital environments and often convert trajectories into coarse skill updates. Directly applying this paradigm to embodied settings is problematic, because a failed task execution may reflect not only incorrect skill content, but also an execution lapse in which the agent fails to follow valid guidance. We propose EmbodiSkill, a training-free framework for embodied skill self-evolution through skill-aware reflection and targeted revision. EmbodiSkill interprets each trajectory with respect to the current skill, uses skill-changing evidence to update the skill body, and uses execution-lapse evidence to preserve and emphasize valid guidance. Experiments on ALFWorld and EmbodiedBench show that EmbodiSkill consistently improves embodied task success. On ALFWorld, EmbodiSkill enables a frozen Qwen3.5-27B executor to reach 93.28% task success, outperforming GPT-5.2 used as a direct agent without skills by 31.58%. These results show that skill-aware self-evolution helps embodied agents accumulate reusable procedural knowledge from their own trajectories. The code and data are available at https://github.com/air-embodied-brain/EmbodiSkill.

1. Introduction

Embodied agents are expected to complete household-like tasks in physical or physically grounded 3D environments, where they must observe the scene, navigate through space, interact with objects, satisfy action preconditions, and handle failed actions [1, 2, 3]. In such tasks, the trajectories generated by different task executions often exhibit recurring procedural structures: an agent may need to locate an object before manipulating it, check whether a container is open before placing it, or adjust its viewpoint when the target is not visible. Following prior work that treats skills, action primitives, or programmatic policies as reusable procedural units for grounding high-level instructions into executable behavior [4, 5, 6, 7], we define an *embodied skill* as a persistent and revisable procedural specification that guides an embodied agent across task executions. In embodied tasks, a skill specifies reusable guidance such as prerequisites, subgoal ordering, object affordances, visual-search strategies, action preconditions, and recovery strategies. Since embodied environments vary in layouts, object states, visibility conditions, and feasible action sequences, an initial skill cannot cover all situations the agent may encounter. Therefore, the central challenge is not only how an embodied agent uses a skill, but how the skill can self-evolve from trajectories into a more complete, accurate, and executable form.

Recent skill self-evolution methods show that agents can improve by extracting, revising, and reusing skills from trajectories [7, 8, 9, 10, 11, 12]. However, many existing methods update skills at a coarse

[*] Ruofei Ju and Xinrui Wang contributed equally to this work.
[†] This work was done while Ruofei Ju and Xinrui Wang were interns at the Institute for AI Industry Research (AIR), Tsinghua University.
[‡] Corresponding authors: Xin Ding (xinding64@mail.ustc.edu.cn) and Ting Cao (tingcao@mail.tsinghua.edu.cn).

清华大学 智能产业研究院
Institute for AI Industry Research, Tsinghua University
2026-7-14

arXiv:2605.10332v2 [cs.AI] 11 Jul 2026

EmbodiSkill：技能感知反思用于自进化具身智能体

朱若飞[1*†] **王新瑞**[2*†] **丁新**[3‡] **杨一帆**[4] **吴昊**[1] **蒋世琪**[4] **张倩溪**[4]**温昊**[5] **李祥宇**[5] **王伟军**[5] **李坤**[5] **刘云欣**[5] **戴海鹏**[1] **王伟**[1] **曹婷**[5‡]

[1] 南京大学 [2] 华中科技大学 [3] 中国科学技术大学 [4] 微软研究院 [5] 人工智能产业研究院（AIR），清华大学

具身智能体可以从指导物体搜索、动作执行和状态变化的各种技能中受益。由于具身环境在布局、对象状态和其他执行因素上存在差异，这些技能必须从任务执行过程中生成的轨迹中自我进化。然而，现有的技能自我进化方法主要是在数字环境中开发的，并且通常将轨迹转换为粗略的技能更新。直接将这种范式应用于具身环境存在问题，因为任务执行失败不仅可能反映不正确的技能内容，还可能反映执行失误，即智能体未能遵循有效指导。我们提出了EmbodiSkill，一个通过技能感知反思和针对性修订进行具身技能自我进化的无训练框架。EmbodiSkill根据当前技能解释每个轨迹，使用技能变化证据来更新技能主体，并使用执行失误证据来保留和强调有效指导。在ALFWorld和EmbodiedBench上的实验表明，EmbodiSkill始终提高了具身任务的成功率。在ALFWorld上，EmbodiSkill使一个冻结的Qwen3.5-27B执行器达到了93.28%的任务成功率，比没有技能直接作为代理使用的GPT-5.2提高了31.58%。这些结果表明，技能感知自进化帮助具身智能体从自己的轨迹中积累可重复的程序性知识。代码和数据可在https://github.com/air-embodied-brain/EmbodiSkill。

1. 简介

具身智能体被期望在物理或物理基础 3D 环境中完成家务类任务，在那里它们必须观察场景、在空间中导航、与物体交互、满足动作前提条件并处理失败的 [1, 2, 3]。在这样的任务中，不同任务执行生成的轨迹经常表现出反复的程序性结构：智能体可能需要在操作物体前找到它，在放置它前检查容器是否打开，或在目标不可见时调整其视角。遵循将技能、动作原语或程序化策略视为可重用程序性单元以将高级指令转化为可执行行为的工作 [4, 5, 6, 7], , 我们定义具身技能 为一种持久且可修订的程序规范，它指导具身智能体跨任务执行。在具身任务中，技能指定可重用的指导，如前提条件、子目标排序、物体可供性、视觉搜索策略、动作前提条件和恢复策略。由于具身环境在布局、对象状态、可见性和可行动作序列方面各不相同，初始技能不能涵盖智能体可能遇到的所有情况。因此，核心挑战不仅在于具身智能体如何使用技能，还在于技能如何从轨迹中自我进化为更完整、准确和可执行的形式。

近期的技能自我进化方法表明，智能体可以通过从轨迹 [7, 8, 9, 10, 11, 12] 中提取、修订和重用技能来改进。然而，许多现有方法在粗略的粒度上更新技能

[*] 朱若飞和王新瑞对这项工作贡献相同。
[†] 这项工作是在朱若飞和王新瑞作为人工智能产业研究院（AIR）清华大学实习生期间完成的。
[‡] 通讯作者：丁新（xinding64@mail.ustc.edu.cn）和曹挺（tingcao@mail.tsinghua.edu.cn）。

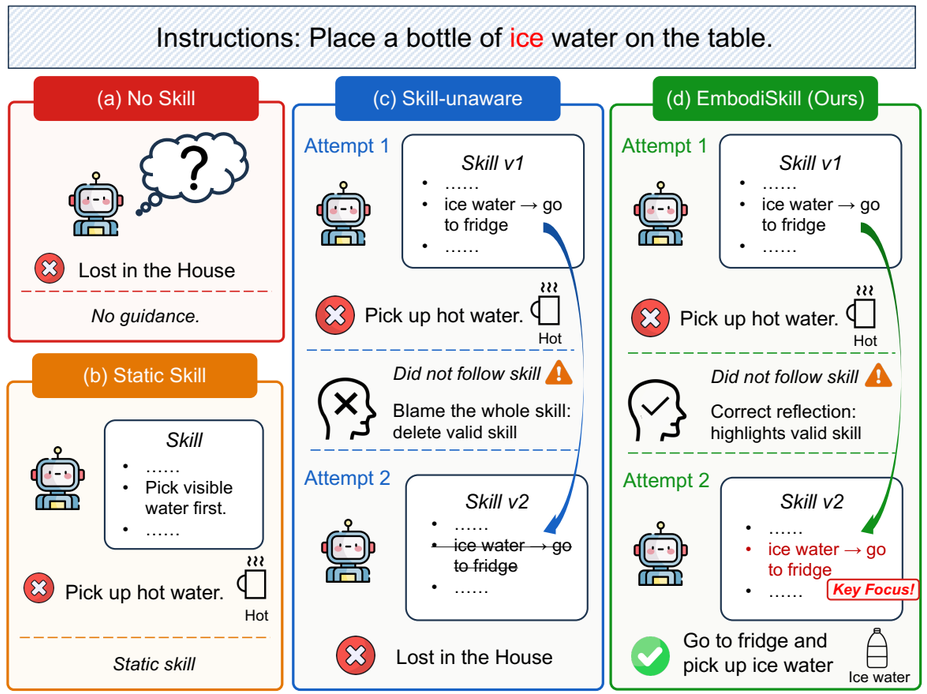


Figure 1 | Motivating example of EmbodiSkill. For the same task instruction, (a) no skill leaves the agent without procedural guidance and causes inefficient exploration, (b) a static skill provides incomplete guidance and cannot adapt from trajectories, and (c) skill-unaware evolution may misinterpret an execution lapse as a skill defect and wrongly revise valid skill content. In contrast, (d) EmbodiSkill reflects on the trajectory against the current skill, preserves valid guidance, and updates the skill to improve later task executions.

granularity: a trajectory is summarized into feedback, and the resulting feedback is used to coarsely update the whole skill, without explicitly identifying which skill content is implicated, why it should be changed, and how it should be changed. This update strategy becomes problematic in embodied environments, where a trajectory outcome is entangled with perception, spatial grounding, object states, action preconditions, and execution reliability [1, 2, 3, 4, 13]. A successful trajectory may reveal useful new skill content or a better way to perform an existing skill, but a failed trajectory does not necessarily mean that the current skill is wrong. It may instead result from the agent failing to follow valid skill content [4, 13, 2]. If such trajectories are converted into coarse skill updates, the evolving skill may accumulate redundant content, overwrite valid guidance, or preserve incorrect prescriptions. Figure 1 illustrates this problem with a motivating example, where skill-unaware evolution misinterprets a failed task execution and incorrectly revises valid skill content.

To address these problems, we propose EmbodiSkill, a framework for embodied skill self-evolution through skill-aware reflection and a skill-aware evolution spiral. Given a trajectory and the current skill,

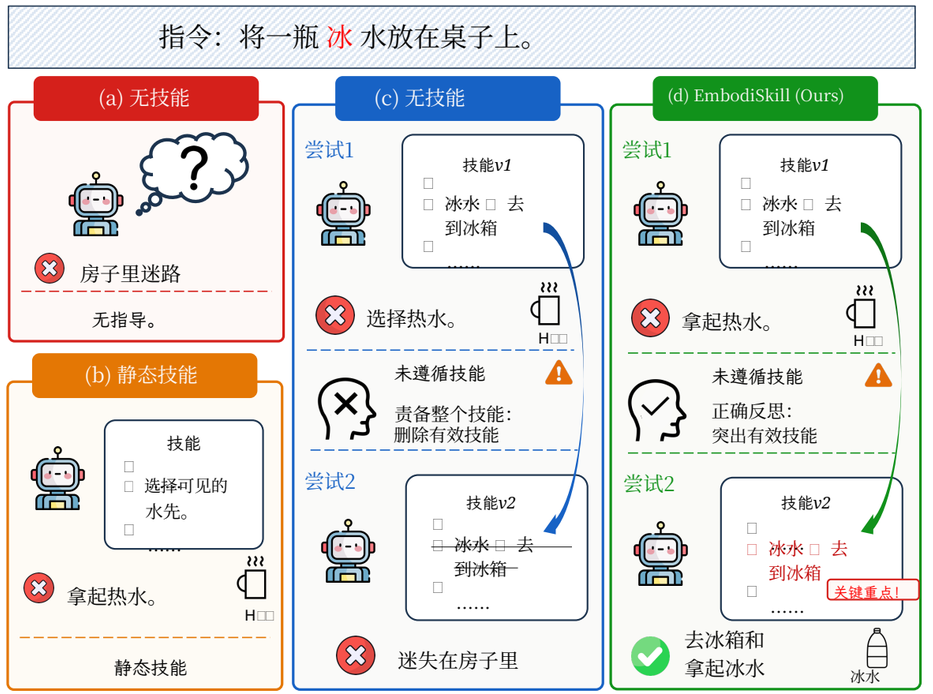


图 1 | EmbodiSkill的激励性示例。对于相同的任务指令，(a)无技能会使代理缺乏程序性指导，导致探索效率低下，(b)静态技能提供不完整的指导，无法从轨迹中适应，以及(c)技能无感知进化可能会将执行失误误解为技能缺陷，并错误地修改有效技能内容。相比之下，(d) EmbodiSkill会针对当前技能对轨迹进行反思，保留有效指导，并更新技能以改进后续任务执行。

粒度：一个轨迹被总结为反馈，生成的反馈被用来粗略地更新整个技能，而没有明确识别哪些技能内容涉及、为什么应该改变以及如何改变。这种更新策略在具身环境中成为问题，其中轨迹结果与感知、空间 grounding、对象状态、动作前提条件和执行可靠性交织在一起 [1, 2, 3, 4, 13]。一个成功的轨迹可能会揭示有用的新技能内容或执行现有技能的更好方法，但一个失败的轨迹并不一定意味着当前技能是错误的。它可能是由智能体未能遵循有效技能内容导致的 [4, 13, 2]。如果这些轨迹被转换为粗略的技能更新，进化的技能可能会积累冗余内容、覆盖有效指导或保留不正确的处方。图 1用一个激励性的例子说明了这个问题，其中技能无感知进化误解了失败的执行任务并错误地修订了有效技能内容。

为解决这些问题，我们提出了EmbodiSkill，一个通过技能感知反思和技能感知进化螺旋实现具身技能自我进化的框架。给定一个轨迹和当前技能，

EmbodiSkill does not convert the trajectory into general feedback for rewriting the whole skill. Instead, skill-aware reflection uses the trajectory to examine the current skill and determine which skill content should be added, optimized, corrected, or preserved. A trajectory can lead to DISCOVERY when it reveals missing skill content, OPTIMIZATION when it suggests a better way to perform an existing skill, SKILL DEFECT when it exposes incorrect, incomplete, or underspecified skill content, or EXECUTION LAPSE when the skill is valid but the agent fails to execute it. The resulting reflections serve as targeted update signals for skill revision. In the skill-aware evolution spiral, justified reflections are integrated into the next skill version, and the revised skill is then used to guide subsequent task executions that generate new trajectories. This creates a closed loop in which trajectories refine the skill, and the refined skill guides later task executions that produce further trajectories for skill evolution. By separating these cases, EmbodiSkill revises only the skill content that should change while preserving valid skill content from unnecessary updates. This selective revision process allows the skill to evolve progressively: each trajectory contributes targeted evidence, and the skill becomes increasingly complete, accurate, and executable over time.

We evaluate EmbodiSkill on embodied and interactive task benchmarks covering visual object interaction, navigation, and household task completion [2, 3]. On ALFWorld, EmbodiSkill enables a local open-weight Qwen3.5-27B executor to achieve 93.28% task success, outperforming GPT-5.2 used as a direct agent by 31.58%. It also exceeds the representative memory-based baseline G-Memory [14] by 25.01%. More importantly, compared with the skill-unaware variant using the same skill evolution model, EmbodiSkill brings a 19.04% relative improvement, demonstrating the importance of skill-aware reflection. These results show that selectively updating the implicated skill content is more effective than coarsely revising the skill, allowing the skill to become progressively more complete, accurate, and executable.

2. Related Work

2.1. Memory-Based Methods for Embodied Agents

In embodied environments, agents often face recurring objects, spatial layouts, action constraints, and failure patterns across task executions, which are reflected in the resulting trajectories. A common way to exploit such recurrence is to adapt memory-based agent methods, which store past trajectories, summaries, or structured notes, and retrieve relevant memory to guide later decision making. For example, Reflexion stores verbal feedback from previous trials as episodic memory [15], while ExpeL extracts reusable lessons from successful and failed trajectories [16]. Recent memory systems such as Mem0, G-Memory, A-MEM, and LangMem further organize agent memory for long-term reuse [17, 14, 18, 19]. However, these methods mainly support trajectory-level reuse rather than skill-level guidance. Retrieved memory records what happened before, but it may remain fragmented or trajectory-specific, rather than forming a persistent skill that specifies how the agent should act in future task executions. This motivates moving beyond retrieved memory toward skills that provide higher-level, persistent, and procedural guidance for future embodied task execution.

2.2. Skills and Skill Self-Evolution for Agents

This higher-level procedural guidance is commonly instantiated as skills. In robotics and physically grounded task settings, prior work often relies on reusable skills, action primitives, or programmatic

EmbodiSkill不会将轨迹转换为用于重写整个技能的通用反馈。相反，技能感知反思利用轨迹来检查当前技能，并确定应添加、优化、纠正或保留哪些技能内容。一个轨迹在揭示缺失技能内容时会导致发现（DISCOVERY），在建议执行现有技能的更好方式时会导致优化（OPTIMIZATION），在暴露不正确、不完整或未充分定义的技能内容时会导致技能缺陷（SKILL DEFECT），或在技能有效但智能体未能执行时会导致执行失误（EXECUTION LAPSE）。生成的反思作为技能修订的靶向更新信号。在技能感知进化螺旋中，合理的反思被整合到下一个技能版本中，然后修订后的技能被用于指导后续的任务执行，这些任务会生成新的轨迹。这形成了一个闭环，其中轨迹精炼技能，而精炼后的技能又指导后续的任务执行，从而产生进一步用于技能进化的轨迹。通过区分这些情况，EmbodiSkill仅修订应更改的技能内容，同时保留有效的技能内容以避免不必要的更新。这种选择性修订过程允许技能逐步进化：每条轨迹都贡献靶向证据，技能随着时间的推移变得越来越完整、准确和可执行。

我们在具身和交互式任务基准测试上评估了EmbodiSkill，这些基准测试涵盖了视觉物体交互、导航和家务任务完成 [2, 3]。在ALFWorld上，EmbodiSkill使本地开放权重Qwen3.5-27B执行器实现了93.28%的任务成功率，比作为直接代理使用的GPT-5.2提高了31.58%。它还超过了代表性的基于记忆的基线G-Memory [14]，提高了25.01%。更重要的是，与使用相同技能进化模型的技能无感知变体相比，EmbodiSkill带来了19.04%的相对提升，证明了技能感知反思的重要性。这些结果表明，选择性地更新相关的技能内容比粗略地修改技能更有效，使技能能够逐步变得更加完整、准确和可执行。

2. 相关工作

2.1. 具身智能体的基于记忆的方法

在具身环境中，智能体在任务执行过程中经常面临重复出现的物体、空间布局、动作约束和失败模式，这些都在生成的轨迹中反映出来。利用这种重复的常见方法之一是适应基于记忆的智能体方法，这些方法存储过去的轨迹、摘要或结构化笔记，并检索相关记忆来指导后续的决策。例如，Reflexion将先前试验中的口头反馈作为情景记忆存储 [15]，，而ExpeL从成功和失败的轨迹中提取可重用的经验教训 [16]。最近的记忆系统，如Mem0、G-Memory、A-MEM和LangMem，进一步组织智能体记忆以供长期重用 [17, 14, 18, 19]。然而，这些方法主要支持轨迹级别的重用，而不是技能级别的指导。检索到的记忆记录了之前发生了什么，但它可能仍然是碎片化的或轨迹特定的，而不是形成一个持续的技能，该技能指定智能体应在未来的任务执行中如何行动。这促使我们从检索到的记忆转向提供更高级、持续和程序性指导的技能，以用于未来的具身任务执行。

2.2. 技能和智能体的技能自我进化

这种高级的程序性指导通常被实例化为技能。在机器人学和物理基础任务设置中，先前的工作通常依赖于可重用技能、动作原语或程序化

policies to connect high-level instructions with executable actions. SayCan selects feasible robotic skills by combining language-model priors with affordance estimates [4]; Code as Policies generates executable policy code by composing perception outputs and control primitives [5]; and ProgPrompt represents household task plans as program-like structures for situated execution [6]. These methods demonstrate the value of skills for embodied task execution, but they mainly focus on selecting, composing, or executing existing skills rather than revising the skill itself from trajectories.

Recent work further studies skill self-evolution and procedural-memory evolution, where agents extract, refine, and reuse skills or reusable procedural knowledge from trajectories and interaction experience. Voyager builds an executable skill library through exploration in an interactive digital world [7]. Trace2Skill distills trajectory-local lessons into transferable agent skills through large-scale trajectory analysis and hierarchical consolidation [10], while EvoSkills studies self-evolving multi-file skill packages through co-evolutionary verification [11]. Other recent work explores related directions such as recursive skill construction, experience-derived executable skills, memory-skill evolution, procedural-memory refinement, and multimodal skill accumulation [20, 12, 21, 8, 9, 22]. These works show that self-evolving skills are a promising paradigm for long-term agent improvement.

However, most existing skill self-evolution methods are not designed for physically grounded embodied environments, where trajectory outcomes are affected by perception, spatial grounding, object states, action preconditions, and execution reliability. As a result, coarse whole-skill update strategies may add redundant content, overwrite valid skill content, or preserve incorrect skill content. EmbodiSkill addresses this gap by making skill self-evolution skill-aware: each trajectory is used to produce targeted revision signals, and justified revisions are integrated through a skill-aware evolution spiral rather than rewriting the skill as a whole.

3. Method

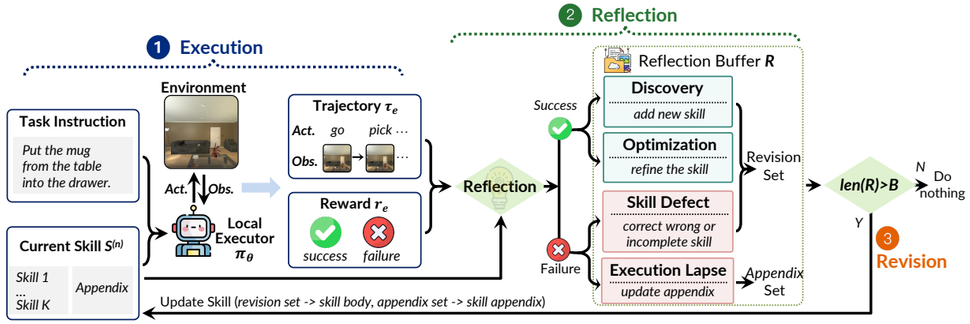


Figure 2 | Overview of EmbodiSkill. The executor uses the current skill to perform embodied tasks and generate trajectories. Skill-aware reflection uses each trajectory to produce targeted reflection records. Accumulated reflections are consolidated into body-level revisions and skill-appendix updates, forming the next skill version. The revised skill then guides subsequent task execution, creating a Skill-Aware Evolution Spiral.

策略来连接高级指令与可执行动作。SayCan通过结合语言模型先验与可供性估计来选择可行的机器人技能 [4]；代码作为策略通过组合感知输出和控制原语来生成可执行策略代码 [5]；以及ProgPrompt将家庭任务计划表示为程序化结构以进行情境执行 [6]。这些方法展示了技能在具身任务执行中的价值，但它们主要关注于选择、组合或执行现有技能，而不是从轨迹中修改技能本身。

近期研究进一步研究了技能自我进化和程序性记忆进化，其中智能体从轨迹和交互经验中提取、精炼和重用技能或可重用程序性知识。Voyager 通过在交互式数字世界中探索构建可执行技能库 [7]。Trace2Skill 通过大规模轨迹分析分层整合，将轨迹局部经验提炼为可转移的代理技能 [10],，而 EvoSkills 通过协同进化验证研究自演化的多文件技能包 [11]。其他近期研究探索了递归技能构建、经验派生的可执行技能、记忆-技能进化、程序性记忆精炼和多模态技能积累等方向 [20, 12, 21, 8, 9, 22]。这些研究表明，自演化的技能是智能体长期改进的一种有前景的范式。

然而，大多数现有的技能自我进化方法并非为物理基础具身环境而设计，在这些环境中，轨迹结果受感知、空间 grounding、对象状态、动作前提条件和执行可靠性影响。因此，粗粒度整体技能更新策略可能会添加冗余内容、覆盖有效技能内容或保留错误技能内容。EmbodiSkill 通过使技能自我进化技能感知来解决这一差距：每个轨迹用于生成目标修正信号，并通过技能感知进化螺旋整合合理的修订，而不是整体重写技能。

3. 方法

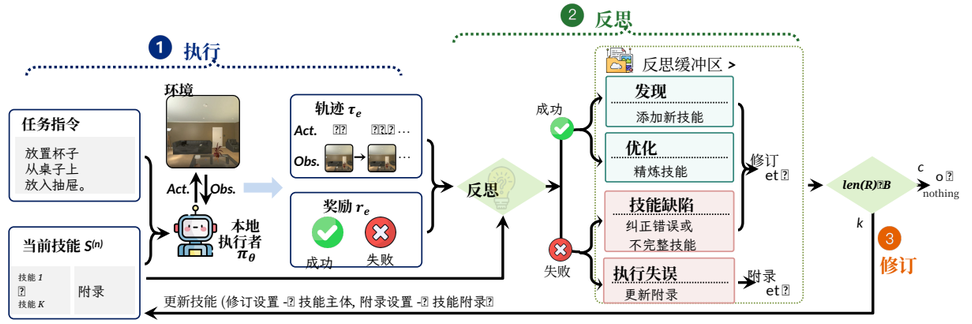


图 2 | EmbodiSkill 概述。执行者使用当前技能执行具身任务并生成轨迹。技能感知反思利用每个轨迹生成目标反思记录。积累的反思被整合为身体级修正和技能附录更新，形成下一个技能版本。修订后的技能随后指导后续任务执行，形成一个技能感知进化螺旋。

3.1. Problem Formulation

Embodied trajectory. We consider an embodied agent interacting with a physical or physically grounded environment under natural-language task instructions. For a task instruction I, the agent observes the environment and produces actions over a finite horizon. We denote the resulting trajectory as

$$\tau = (I, o_1, a_1, \ldots, o_T, a_T, r), \tag{1}$$

where o_t is the observation at step t, a_t is the action, T is the trajectory length, and $r \in \{0, 1\}$ is the final task success signal.

Skill-guided execution. At evolution step n, the agent is guided by a skill $S^{(n)}$, a persistent and revisable procedural specification for embodied task execution. We represent the skill as

$$S^{(n)} = \left(S^{(n)}_{\text{body}}, S^{(n)}_{\text{app}}\right), \tag{2}$$

where $S^{(n)}_{\text{body}}$ contains the main prescriptive skill content for future task execution, and $S^{(n)}_{\text{app}}$ is a skill appendix that emphasizes valid content in $S^{(n)}_{\text{body}}$. The appendix does not introduce new skill rules; it only highlights existing skill content that should be followed carefully during execution.

Given the current skill, the executor π_θ generates actions according to

$$a_t \sim \pi_\theta(\cdot \mid I, S^{(n)}, h_t), \tag{3}$$

where $h_t = (o_1, a_1, \ldots, o_t)$ denotes the within-trajectory history. The executor parameters θ are kept fixed; all improvement across task executions is externalized into the evolving skill.

Skill evolution model. The executor π_θ uses the current skill to perform embodied tasks and produce trajectories. In contrast, the skill evolution model F operates after trajectories are completed. We use F as a unified notation for the model calls used in skill-aware reflection, reflection consolidation, skill-body revision, and skill-appendix update; these calls share the same underlying model but use different prompts and output formats.

Skill self-evolution objective. The goal of embodied skill self-evolution is to construct a sequence of skills

$$S^{(0)} \to S^{(1)} \to \cdots \to S^{(N)} \tag{4}$$

such that later skills lead to higher task success when used by the same executor. Formally, we aim to improve

$$J(S; \pi_\theta) = \mathbb{E}_{I,\mathcal{E}}\left[r(\tau)\right], \quad \tau = \text{Execute}(\pi_\theta, I, \mathcal{E}, S), \tag{5}$$

where $\mathcal{E}$ denotes the embodied environment distribution.

3.1. 问题公式化

具身轨迹。 我们考虑一个具身智能体在与物理基础环境交互时遵循自然语言任务指令。对于任务指令 I，智能体观察环境并在有限时间范围内产生动作。我们将产生的轨迹表示为

$$\tau = (I, o_1, a_1, \ldots, o_T, a_T, r), \tag{1}$$

其中 o_t 是步骤 t 的观察结果， a_t 是动作， T 是轨迹长度，而 $r \in \{0,1\}$ 是最终的任务成功信号。

技能引导执行。 在进化步骤 n，智能体由一个技能 $S_{(n)}$ 引导，这是一个用于具身任务执行的持久且可修订的程序规范。我们将技能表示为

$$S^{(n)} = \left(S^{(n)}_{\text{body}}, S^{(n)}_{\text{app}}\right), \tag{2}$$

其中 $S_{(n)\ \text{身体}}$ 包含用于未来任务执行的主要技能规范内容，而 $S_{(n)\text{附录}}$ 是一个技能附录，它强调在 $S_{(n)\ \text{身体}}$中的有效内容。附录不会引入新的技能规则；它仅突出在执行过程中需要仔细遵循的现有技能内容。

给定当前技能，执行者 π_θ 根据

$$a_t \sim \pi_\theta(\cdot \mid I, S^{(n)}, h_t), \tag{3}$$

其中 $h_t = (o_1, a_1, \ldots, o_t)$ 表示轨迹内历史。执行者参数 θ 保持固定；所有任务执行中的改进都外部化为进化的技能。

技能进化模型。 执行者 π_θ 使用当前技能执行具身任务并产生轨迹。相比之下，技能进化模型 F 在轨迹完成后运行。我们使用 F 作为技能感知反思、反思整合、技能体修订和技能附录更新中使用的模型调用的统一符号；这些调用共享相同的底层模型，但使用不同的提示和输出格式。

技能自我进化目标。 具身技能自我进化的目标是构建一个技能序列

$$S^{(0)} \to S^{(1)} \to \cdots \to S^{(N)} \tag{4}$$

使得当由相同的执行者使用时，后续技能能导致更高的任务成功。形式上，我们旨在改进

$$J(S; \pi_\theta) = \mathbb{E}_{I,\mathcal{E}}\left[r(\tau)\right], \quad \tau = \text{Execute}(\pi_\theta, I, \mathcal{E}, S), \tag{5}$$

where E 表示具身环境分布。

Algorithm 1: EmbodiSkill

Input: Initial skill $S^{(0)} = (S^{(0)}_{\text{body}}, S^{(0)}_{\text{app}})$, executor π_θ, skill evolution model F, task stream $\mathcal{I}$, revision interval B, maximum reflections per trajectory K
Output: Evolved skill S

$(S_{\text{body}}, S_{\text{app}}) \leftarrow S^{(0)}$;
$S \leftarrow (S_{\text{body}}, S_{\text{app}})$;
$\mathcal{R} \leftarrow \emptyset$; `// reflection buffer`
foreach *task instruction* $I \in \mathcal{I}$ **do**
  $\tau \leftarrow \textsc{Execute}(\pi_\theta, I, S)$;
  $\mathcal{R}_\tau \leftarrow \textsc{SkillAwareReflect}(F, \tau, S, K)$;
  if $\mathcal{R}_\tau \neq \emptyset$ **then**
    $\mathcal{R} \leftarrow \mathcal{R} \cup \mathcal{R}_\tau$;
  if $|\mathcal{R}| \geq B$ **then**
    $(\mathcal{R}_{\text{disc}}, \mathcal{R}_{\text{opt}}, \mathcal{R}_{\text{def}}, \mathcal{R}_{\text{lap}}) \leftarrow \textsc{PartitionByType}(\mathcal{R})$;
    $\widetilde{\mathcal{R}}_{\text{rev}} \leftarrow \textsc{ConsolidateRevisions}(F, S_{\text{body}}, \mathcal{R}_{\text{disc}}, \mathcal{R}_{\text{opt}}, \mathcal{R}_{\text{def}})$;
    $S_{\text{body}} \leftarrow \textsc{ReviseSkillBody}(F, S_{\text{body}}, \widetilde{\mathcal{R}}_{\text{rev}})$; `// targeted body edits`
    $S_{\text{app}} \leftarrow \textsc{UpdateSkillAppendix}(F, S_{\text{body}}, S_{\text{app}}, \mathcal{R}_{\text{lap}})$; `// update appendix only`
    $S \leftarrow (S_{\text{body}}, S_{\text{app}})$;
    $\mathcal{R} \leftarrow \emptyset$;
return S;

3.2. EmbodiSkill

EmbodiSkill implements embodied skill self-evolution as a skill-aware reflection and revision loop, as shown in Figure 2 and summarized in Algorithm 1. At evolution step n, the executor uses the current skill $S^{(n)}$ to perform embodied tasks, and each task execution produces a trajectory. After each trajectory, a skill evolution model uses the resulting trajectory to examine $S^{(n)}$ and produces up to K targeted reflections, or no reflection when the trajectory does not provide reliable evidence for skill revision. Each valid reflection specifies the trajectory evidence, the corresponding revision directive, and either the implicated existing skill content or newly proposed skill content.

After a fixed number of valid reflections are accumulated, EmbodiSkill consolidates them before revising the skill. Reflections that propose body-level updates are consolidated to remove redundancy and resolve conflicts before updating S_{body}. Reflections attributed to execution lapses do not revise the skill body; instead, after S_{body} is updated, they are used together with the updated skill body and the previous skill appendix to produce a new S_{app}. The final revision step therefore yields the next skill version $S^{(n+1)} = (S^{(n+1)}_{\text{body}}, S^{(n+1)}_{\text{app}})$, where substantive changes to the skill body are grounded in skill-aware revision signals, while the skill appendix highlights valid skill content that should be followed carefully during later execution. The revised skill then guides subsequent task execution, forming a Skill-Aware Evolution Spiral.

算法1： EmbodiSkill

输入： 初始技能 $S^{(0)} = (S^{(0)}_{\text{body}}, S^{(0)}_{\text{应用}})$, 执行者 π_θ, 技能进化模型 F, 任务流I, 修订间隔 B, 每个轨迹的最大反思次数 K
输出： 进化技能 S

$(S_{\text{身体}}, S_{\text{app}}) \leftarrow S^{(0)}$;
$S \leftarrow (S_{\text{身体}}, S_{\text{app}})$;
$\mathcal{R} \leftarrow \emptyset$; `// reflection buffer`
foreach任务指令 $I \in \mathcal{I}$ **执行**
  $\tau \leftarrow$ 执行(π_θ, I, S);
  $\mathcal{R}_\tau \leftarrow$ SkillAwareReflect(F, τ, S, K);
  如果 $\mathcal{R}_\tau \neq \emptyset$**则**
    $\mathcal{R} \leftarrow \mathcal{R} \cup \mathcal{R}_\tau$;
  如果 $|\mathcal{R}| \geq B$ **则**
    $(\mathcal{R}_{\text{disc}}, \mathcal{R}_{\text{opt}}, \mathcal{R}_{\text{def}}, \mathcal{R}_{\text{lap}}) \leftarrow$ 按类型划分$(\mathcal{R})$;
    $\widetilde{\mathcal{R}}_{\text{rev}} \leftarrow$ 整合修订$(F, S_{\text{身体}}, \mathcal{R}_{\text{disc}}, \mathcal{R}_{\text{opt}}, \mathcal{R}_{\text{def}})$;
    $S_{\text{身体}} \leftarrow$ 修订技能主体$(F, S_{\text{身体}}, \widetilde{\mathcal{R}}_{\text{rev}})$; `// targeted body edits`
    $S_{\text{app}} \leftarrow$ 更新技能附录$(F, S_{\text{身体}}, S_{\text{app}}, \mathcal{R}_{\text{lap}})$; `// update appendix only`
    $S \leftarrow (S_{\text{身体}}, S_{\text{app}})$;
    $\mathcal{R} \leftarrow \emptyset$;
return S;

3.2. EmbodiSkill

EmbodiSkill 实现了具身技能自我进化，将其作为技能感知反思和修订循环，如图 2 和算法 1 所示。在进化步骤 n，执行者使用当前技能 $S^{(n)}$ 执行具身任务，每个任务执行都会产生一个轨迹。在每个轨迹之后，技能进化模型使用生成的轨迹进行检查 $S^{(n)}$，并产生最多 K 个目标反思，或者当轨迹不提供可靠的技能修订证据时则不产生反思。每个有效的反思指定了轨迹证据、相应的修订指令，以及涉及现有技能内容或新提出的技能内容。

积累了一定数量的有效反思后，EmbodiSkill 会将它们整合起来，然后再对技能进行修订。提出身体级别更新的反思会被整合以消除冗余并解决冲突，然后再更新 $S_{\text{身体}}$。归因于执行失误的反思不会修订技能主体；相反，在 $S_{\text{身体}}$ 更新后，它们会与更新的技能主体和之前的技能附录一起用于生成新的 S_{app}</style}。因此，最终的修订步骤会产生下一个技能版本 $(n+1) = (S^{(n+1)}_{\text{身体}}, S^{(n+1)}_{\text{app}})$，其中对技能主体的实质性更改基于技能感知修订信号，而技能附录突出显示有效技能内容，这些内容在后续执行过程中应谨慎遵循。修订后的技能指导后续任务执行，形成技能感知进化螺旋。

3.2.1. Skill-Aware Reflection

Skill-aware reflection determines how a trajectory should be interpreted with respect to the current skill. Given a trajectory τ and the current skill $S^{(n)} = (S^{(n)}_{\text{body}}, S^{(n)}_{\text{app}})$, the skill evolution model produces a set of reflection records:

$$\mathcal{R}_\tau = F(\tau, S^{(n)}, K) = \{\rho_i\}_{i=1}^{m_\tau}, \quad 0 \le m_\tau \le K, \tag{6}$$

where K is the maximum number of reflections allowed for one trajectory. When the trajectory does not provide reliable evidence for skill revision, $m_\tau = 0$ and no reflection is produced.

Each reflection record ρ_i contains a reflection type c_i, trajectory evidence e_i, and an update directive d_i. For reflection types that modify or emphasize existing skill content, the record also contains a target skill content b_i that refers to a specific part of $S^{(n)}_{\text{body}}$. The reflection type is chosen according to the trajectory outcome. For a successful trajectory, the reflection type is chosen from

$$c_i \in \{\textsc{Discovery}, \textsc{Optimization}\}, \quad r = 1, \tag{7a}$$

whereas for a failed trajectory, the reflection type is chosen from

$$c_i \in \{\textsc{SkillDefect}, \textsc{ExecutionLapse}\}, \quad r = 0. \tag{7b}$$

DISCOVERY. A Discovery reflection indicates that the trajectory reveals useful skill content not covered by the current skill body. It does not require a target skill content, because it proposes new skill content rather than modifying existing content. Its directive d_i specifies the new skill content to be considered during revision.

OPTIMIZATION. An Optimization reflection indicates that an existing target skill content is valid, but the trajectory suggests a more effective way to perform it. It must identify a target skill content b_i and provide a revised version or modification directive for that target skill content.

SKILL DEFECT. A SkillDefect reflection indicates that an existing target skill content is incorrect, incomplete, or underspecified. It must identify the problematic target skill content b_i, provide trajectory evidence for the defect, and propose corrected skill content.

EXECUTION LAPSE. An ExecutionLapse reflection indicates that the target skill content is valid, but the executor fails to follow it in the trajectory. It must identify the valid target skill content b_i and describe the deviation between what the skill requires and what the executor actually did. Its directive d_i is not a body-level revision; instead, it produces appendix content for S_{app}, reminding the executor to follow the corresponding valid skill content carefully during later task execution.

Thus, skill-aware reflection differs from trajectory summarization. It does not convert a trajectory into general feedback for rewriting the whole skill. Instead, it uses the trajectory to determine whether the trajectory supports adding new skill content, optimizing or correcting existing skill content, or updating the skill appendix, and produces structured signals for the revision procedure described next.

3.2.1. 技能感知反思

技能感知反思决定了轨迹应如何相对于当前技能进行解释。给定一个轨迹 τ 和当前技能 $S^{(n)} = (S^{(n)}$ 身体, $S^{(n)}$app$)$，技能进化模型会生成一组反思记录：

$$\mathcal{R}_\tau = F(\tau, S^{(n)}, K) = \{\rho_i\}_{i=1}^{m_\tau}, \quad 0 \le m_\tau \le K, \tag{6}$$

其中 K 是一个轨迹允许的最大反思次数。当轨迹没有为技能修订提供可靠证据时， $m_\tau = 0$ 并且不产生反思。

每个反思记录 ρ_i 包含一个反思类型 c_i、轨迹证据 e_i 和一个更新指令 d_i。对于修改或强调现有技能内容的反思类型，记录还包含一个目标技能内容 b_i，它引用了 $S^{(n)}$ 身体 的特定部分。反思类型是根据轨迹结果选择的。对于成功的轨迹，反思类型从

$$c_i \in \{\textsc{Discovery}, \textsc{Optimization}\}, \quad r = 1, \tag{7a}$$

而失败的轨迹，反思类型从

$$c_i \in \{\textsc{SkillDefect}, \textsc{ExecutionLapse}\}, \quad r = 0. \tag{7b}$$

发现。 一个发现型反思表明轨迹揭示了对当前技能主体没有覆盖的有用技能内容。它不需要目标技能内容，因为它提出的是新技能内容而不是修改现有内容。其指令 d_i 指定了在修订过程中要考虑的新技能内容。

优化。 一个优化型反思表明一个现有的目标技能内容是有效的，但轨迹建议更有效的方式来执行它。它必须识别一个目标技能内容 b_i 并为该目标技能内容提供修订版本或修改指令。

技能缺陷。 一个SkillDefect反思表示现有的目标技能内容不正确、不完整或未充分定义。它必须识别有问题的目标技能内容 b_i，提供缺陷的轨迹证据，并提出修正后的技能内容。

执行滞后。 一个ExecutionLapse反思表示目标技能内容是有效的，但执行者未能在其轨迹中遵循它。它必须识别有效的目标技能内容 b_i，并描述技能要求与执行者实际行为之间的偏差。其指令 d_i不是身体级别的修订；相反，它为 S_{app}生成附录内容，提醒执行者在后续任务执行时仔细遵循相应的有效技能内容。

因此，技能感知反思与轨迹总结不同。它不会将一个轨迹转换为用于重写整个技能的一般反馈。相反，它使用轨迹来确定轨迹是否支持添加新的技能内容、优化或纠正现有的技能内容，或更新技能附录，并为下一步描述的修订程序产生结构化信号。

3.2.2. Skill Revision

Skill revision converts accumulated skill-aware reflections into the next skill version. Once the reflection buffer reaches the revision interval B, EmbodiSkill partitions the buffered reflections by type:

$$(\mathcal{R}_{\text{disc}}, \mathcal{R}_{\text{opt}}, \mathcal{R}_{\text{def}}, \mathcal{R}_{\text{lap}}) = \textsc{PartitionByType}(\mathcal{R}), \tag{8}$$

where $\mathcal{R}_{\text{disc}}$, $\mathcal{R}_{\text{opt}}$, $\mathcal{R}_{\text{def}}$, and $\mathcal{R}_{\text{lap}}$ correspond to Discovery, Optimization, SkillDefect, and ExecutionLapse, respectively.

Consolidating body-level revision signals. The skill body is revised only from Discovery, Optimization, and SkillDefect reflections. Before editing the skill body, EmbodiSkill consolidates these reflections:

$$\widetilde{\mathcal{R}}_{\text{rev}} = F\left(S_{\text{body}}^{(n)}, \mathcal{R}_{\text{disc}}, \mathcal{R}_{\text{opt}}, \mathcal{R}_{\text{def}}\right). \tag{9}$$

This consolidation step turns multiple local reflections into a consistent set of revision signals. It removes redundant reflections, merges overlapping suggestions, groups target-specific modifications by their target skill contents, and resolves conflicts among signals when possible. If a signal is misplaced or conflicts cannot be reliably resolved, the model either reassigns it to a more appropriate revision type or discards it, rather than forcing uncertain changes into the skill.

Revising the skill body. The consolidated revision signals are then used to update the skill body:

$$S_{\text{body}}^{(n+1)} = F\left(S_{\text{body}}^{(n)}, \widetilde{\mathcal{R}}_{\text{rev}}\right). \tag{10}$$

This step uses the skill evolution model as a constrained editor rather than a free-form rewriter. The editor revises the skill body according to the consolidated revision set $\widetilde{\mathcal{R}}_{\text{rev}}$: Discovery adds new skill content, while Optimization and SkillDefect modify their identified target skill contents. Skill content not implicated by $\widetilde{\mathcal{R}}_{\text{rev}}$ is kept unchanged in substance. The editor may still perform limited consistency edits, such as removing redundancy, normalizing format, or resolving local conflicts among the affected skill content. Thus, the revision updates targeted skill content while avoiding coarse rewriting of the whole skill.

Updating the skill appendix. After the skill body has been revised, EmbodiSkill updates the appendix part using ExecutionLapse reflections:

$$S_{\text{app}}^{(n+1)} = F\left(S_{\text{body}}^{(n+1)}, S_{\text{app}}^{(n)}, \mathcal{R}_{\text{lap}}\right). \tag{11}$$

Unlike the body-level revision step, this update does not introduce, delete, or rewrite skill rules in S_{body}. Instead, it organizes execution-lapse evidence into appendix content anchored to the updated skill body. Each appendix item highlights a valid skill that the executor failed to follow, so that later executions pay closer attention to that skill. The appendix update can merge duplicate appendix items, remove obsolete appendix items that no longer correspond to the updated skill body, and incorporate new execution-lapse reflections.

3.2.2. 技能修订

技能修订将积累的技能感知反思转换为下一个技能版本。一旦反思缓冲区达到修订间隔 B，EmbodiSkill 按类型对缓冲的反思进行分区：

$$(\mathcal{R}_{\text{disc}}, \mathcal{R}_{\text{opt}}, \mathcal{R}_{\text{def}}, \mathcal{R}_{\text{lap}}) = \textsc{PartitionByType}(\mathcal{R}), \tag{8}$$

其中 R_{disc}, R^{opt}, R_{def}, 和 R^{lap} 分别对应发现、优化、技能缺陷和执行滞后。

整合身体级别的修订信号。 技能主体仅从发现、优化和技能缺陷型反思中进行修订。在编辑技能主体之前，EmbodiSkill 整合这些反思：

$$\widetilde{\mathcal{R}}_{\text{rev}} = F\left(S_{\text{body}}^{(n)}, \mathcal{R}_{\text{disc}}, \mathcal{R}_{\text{opt}}, \mathcal{R}_{\text{def}}\right). \tag{9}$$

这一整合步骤将多个局部反思转化为一致的一组修订信号。它移除冗余的反思，合并重叠的建议，按其目标技能内容对目标特定修改进行分组，并在可能的情况下解决信号之间的冲突。如果一个信号位置不当或冲突无法可靠解决，模型要么将其重新分配给更合适的修订类型，要么将其丢弃，而不是将不确定的改动强加到技能中。

修订技能主体。 整合后的修订信号随后用于更新技能主体：

$$S_{\text{body}}^{(n+1)} = F\left(S_{\text{body}}^{(n)}, \widetilde{\mathcal{R}}_{\text{rev}}\right). \tag{10}$$

这一步骤将技能进化模型用作约束编辑器而不是自由形式重写器。编辑器~根据整合的修订集 R_{rev} 修订技能主体：发现添加新的技能内容，而优化和技能缺陷修改其识别的目标技能内容。未受 R_{rev} 涉及的技能内容在实质上保持不变。编辑器仍可能执行有限的格式一致性编辑，例如移除冗余、规范化格式或解决受影响技能内容之间的局部冲突。因此，修订更新了目标技能内容，同时避免了粗略重写整个技能。

更新技能附录。 技能主体修订后，EmbodiSkill 使用执行滞后型反思更新附录部分：

$$S_{\text{app}}^{(n+1)} = F\left(S_{\text{body}}^{(n+1)}, S_{\text{app}}^{(n)}, \mathcal{R}_{\text{lap}}\right). \tag{11}$$

与身体级别的修订步骤不同，此更新不会在 $S_{\text{身体}}$中引入、删除或重写技能规则。相反，它将执行滞后证据组织为锚定到更新后的技能主体的附录内容。每个附录项突出显示执行者未能遵循的有效技能，以便后续执行更关注该技能。附录更新可以合并重复的附录项，删除不再与更新后的技能主体对应的过时附录项，并纳入新的执行滞后反思。

The next skill version is therefore

$$S^{(n+1)} = \left(S_{\text{body}}^{(n+1)}, S_{\text{app}}^{(n+1)}\right). \tag{12}$$

This design separates body-level skill revision from skill-appendix update: DISCOVERY, OPTIMIZATION, and SKILLDEFECT reflections can change the skill body, while EXECUTIONLAPSE reflections only affect the appendix part. As a result, EmbodiSkill updates the skill selectively while reducing the risk of corrupting valid skill content.

3.2.3. Skill-Aware Evolution Spiral

The Skill-Aware Evolution Spiral describes how EmbodiSkill turns repeated skill-aware revisions into progressive skill improvement. At evolution step n, the executor uses the current skill $S^{(n)} = (S_{\text{body}}^{(n)}, S_{\text{app}}^{(n)})$ to perform embodied tasks. Each task execution produces a trajectory, which is reflected against the current skill to generate revision signals. These signals update the skill into $S^{(n+1)} = (S_{\text{body}}^{(n+1)}, S_{\text{app}}^{(n+1)})$. The revised skill is then used for subsequent task execution, producing new trajectories that may expose missing skill content, better execution strategies, skill defects, or execution lapses. In this way, the skill shapes future task execution, and the resulting trajectories provide new evidence for further skill evolution.

The spiral is driven by the interaction between the evolving skill and the trajectories it produces. When S_{body} is expanded, optimized, or corrected, the executor may handle task situations that were previously difficult, producing trajectories that expose new opportunities for further skill improvement. When S_{app} is updated from execution lapses, the executor is reminded to follow valid skill content more carefully during subsequent task execution. Across evolution steps, the skill therefore accumulates targeted improvements: S_{body} becomes more complete and accurate, while S_{app} makes valid skill content more salient during execution, improving the executability of the overall skill.

4. Experiments

4.1. Experimental Setup

Benchmarks and metrics. We evaluate EmbodiSkill on three embodied task benchmarks: ALFWorld [2], EmbodiedBench-Habitat, and EmbodiedBench-Navigation [3]. ALFWorld contains household task completion problems in interactive environments, involving object search, navigation, container interaction, and object state changes. EmbodiedBench-Habitat evaluates visual object interaction in 3D environments, while EmbodiedBench-Navigation evaluates visual navigation. In our split, ALFWorld contains 3,553 training tasks and 134 test tasks; EmbodiedBench-Habitat contains 1,000 training tasks and six test subsets with 50 tasks each; and EmbodiedBench-Navigation contains 1,000 training tasks and five test subsets with 60 tasks each. We report task success rate for all benchmarks.

Skill evolution protocol. EmbodiSkill evolves skills from trajectories collected on training tasks and evaluates the evolved skill on held-out test tasks. We set the maximum number of reflections per trajectory to $K = 1$ and perform 10 skill revision stages unless otherwise specified. Training tasks are sampled in randomized order, and the training stream is reshuffled when additional trajectories are required. During test evaluation, the evolved skill is fixed.

因此下一个技能版本是

$$S^{(n+1)} = \left(S_{\text{body}}^{(n+1)}, S_{\text{app}}^{(n+1)}\right). \tag{12}$$

此设计将身体级技能修订与技能附录更新分离：发现、优化和技能缺陷型反思可以改变技能主体，而执行滞后型反思仅影响附录部分。因此，EmbodiSkill选择性地更新技能，同时降低损坏有效技能内容的风险。

3.2.3. 技能感知进化螺旋

技能感知进化螺旋描述了EmbodiSkill如何将重复的技能感知修订转化为渐进的技能改进。在进化步骤 n，执行者使用当前技能 $S^{(n)} = (S^{(n)}_{\text{身体}}, S^{(n)}_{\text{附录}})$ 来执行具身任务。每次任务执行都会产生一个轨迹，该轨迹与当前技能进行反思以生成修订信号。这些信号将技能更新为 $S^{(n+1)} = (S^{(n+1)}_{\text{身体}}, S^{(n+1)}_{\text{附录}})$。修订后的技能随后用于后续任务执行，产生新的轨迹，这些轨迹可能会暴露缺失的技能内容、更好的执行策略、技能缺陷或执行滞后。通过这种方式，技能塑造了未来的任务执行，而由此产生的轨迹为进一步的技能进化提供了新的证据。

螺旋由进化的技能与其产生的轨迹之间的交互驱动。当 $S_{\text{身体}}$ 被扩展、优化或纠正时，执行者可以处理以前难以处理的任务情况，产生暴露进一步技能改进新机会的轨迹。当 S_{app} 从执行失误更新时，执行者被提醒在后续任务执行期间更仔细地遵循有效技能内容。因此，在进化步骤中，技能积累了有针对性的改进： $S_{\text{身体}}$ 变得更加完整和准确，而 S_{app} 在执行期间使有效技能内容更加突出，提高了整体技能的可执行性。

4. 实验

4.1. 实验设置

基准测试和指标。 我们在三个具身任务基准测试上评估 EmbodiSkill：ALFWorld [2]，EmbodiedBench-Habitat，和 EmbodiedBench-Navigation [3]。 ALFWorld 包含交互环境中的家庭任务完成问题，涉及物体搜索、导航、容器交互和物体状态变化。EmbodiedBench-Habitat 评估 3D 环境中的视觉物体交互，而 EmbodiedBench-Navigation 评估视觉导航。在我们的划分中，ALFWorld 包含 3,553 个训练任务和 134 个测试任务；EmbodiedBench-Habitat 包含 1,000 个训练任务和六个每个包含 50 个任务的测试子集；而 EmbodiedBench-Navigation 包含 1,000 个训练任务和五个每个包含 60 个任务的测试子集。我们报告所有基准测试的任务成功率。

技能进化协议。 EmbodiSkill 从训练任务上收集的轨迹中进化技能，并在保留的测试任务上评估进化后的技能。我们设置每个轨迹的最大反思次数为 $K = 1$，除非另有指定，否则执行 10 个技能修订阶段。训练任务以随机顺序采样，当需要额外轨迹时，训练流会被重新洗牌。在测试评估期间，进化后的技能是固定的。

Table 1 | Task success rate (%) on ALFWorld. We report overall success and six ALFWorld subtask categories. Bold indicates the best result in each column; underline indicates the second-best result on the overall score.

Method	Overall	ALFWorld Subtasks					
		Put	Clean	Heat	Cool	Examine	Puttwo
Closed-source direct agents							
GPT-5.2	70.89	87.50	67.74	56.52	76.19	83.33	52.94
Gemini-3-flash	82.09	91.67	83.87	65.22	85.71	83.33	82.35
Executor: Qwen2.5-14B-Instruct							
Memory-based baselines							
No memory	46.27	33.33	51.61	26.09	85.71	72.22	5.88
Mem0	12.69	0	0	4.35	0	88.89	0
G-Memory	67.16	50.00	83.87	69.57	85.71	83.33	17.65
LangMem	36.57	12.50	32.26	26.09	71.43	83.33	0.00
EmbodiSkill (Ours)							
EmbodiSkill w/ GPT-5.2	86.57	91.67	**96.77**	73.91	**95.24**	83.33	70.59
EmbodiSkill w/ Gemini-3-flash	85.82	79.17	93.55	73.91	90.48	**100.00**	76.47
Executor: Qwen3.5-27B							
Memory-based baselines							
No memory	61.19	66.67	51.61	65.22	76.19	72.22	35.29
Mem0	64.18	62.50	61.29	56.52	85.71	72.22	47.06
G-Memory	74.62	62.50	77.42	**82.61**	85.71	83.33	52.94
LangMem	62.69	62.50	61.29	52.17	76.19	72.22	52.94
EmbodiSkill (Ours)							
EmbodiSkill w/ GPT-5.2	**93.28**	**95.83**	**96.77**	73.91	**95.24**	**100.00**	**100.00**
EmbodiSkill w/ Gemini-3-flash	<u>87.31</u>	**95.83**	93.55	69.57	**95.24**	83.33	82.35

Models. For ALFWorld, we instantiate the executor π_θ with Qwen2.5-14B-Instruct [23] and Qwen3.5-27B [24]. For EmbodiedBench-Habitat and EmbodiedBench-Navigation, Qwen3-VL-8B-Instruct and Qwen3-VL-32B-Instruct as executors [25]. The skill evolution model F is instantiated with GPT-5.2 [26] or Gemini-3-flash [27]. All executor parameters are kept fixed during skill evolution, so performance gains come from the evolving skill rather than model parameter updates.

Baselines. We compare EmbodiSkill with two groups of baselines in the main results. First, closed-source direct agents use GPT-5.2 or Gemini-3-flash directly for task execution without an evolving skill. Second, memory-based methods, including Mem0 [17], G-Memory [14], and LangMem [19], store and retrieve trajectory-level information to guide task execution. For methods using the same local executor, the executor is kept identical so that performance differences reflect the external skill or memory mechanism rather than changes in the executor.

4.2. Main Results

Table 1 and Table 2 report the main results on ALFWorld and EmbodiedBench. Overall, EmbodiSkill improves task success by evolving an external skill from training trajectories, showing stronger performance

表 1 | 在 ALFWorld 上的任务成功率 (%)。我们报告了总体成功率和六个 ALFWorld 子任务类别。粗体表示每列的最佳结果；下划线表示在总体得分上的第二好的结果。

方法	总体	ALFWorld 子任务					
		Put	清洁	Heat	冷却	检查	放置两件物品
闭源直接代理							
GPT-5.2	70.89	87.50	67.74	56.52	76.19	83.33	52.94
Gemini-3-flash	82.09	91.67	83.87	65.22	85.71	83.33	82.35
执行者: Qwen2.5-14B-Instruct							
基于记忆的基线							
无记忆	46.27	33.33	51.61	26.09	85.71	72.22	5.88
Mem0	12.69	0	0	4.35	0	88.89	0
G-Memory	67.16	50.00	83.87	69.57	85.71	83.33	17.65
LangMem	36.57	12.50	32.26	26.09	71.43	83.33	0.00
EmbodiSkill (我们)							
EmbodiSkill w/ GPT-5.2	86.57	91.67	96.77	73.91	**95.24**	83.33	70.59
EmbodiSkill w/ Gemini-3-flash	85.82	79.17	93.55	73.91	90.48	**100.00**	76.47
执行者: Qwen3.5-27B							
基于记忆的基线							
无记忆	61.19	66.67	51.61	65.22	76.19	72.22	35.29
Mem0	64.18	62.50	61.29	56.52	85.71	72.22	47.06
G-Memory	74.62	62.50	77.42	82.61	85.71	83.33	52.94
LangMem	62.69	62.50	61.29	52.17	76.19	72.22	52.94
EmbodiSkill (我们)							
EmbodiSkill w/ GPT-5.2	**93.28**	**95.83**	96.77	73.91	**95.24**	**100.00**	**100.00**
EmbodiSkill w/ Gemini-3-flash	<u>87.31</u>	**95.83**	93.55	69.57	**95.24**	83.33	82.35

模型。对于 ALFWorld，我们使用执行者 π_θ 实例化 Qwen2.5-14B-Instruct [23] 和 Qwen3.5- 27B [24]。对于 EmbodiedBench-Habitat 和 EmbodiedBench-Navigation，使用 Qwen3-VL-8B-Instruct 和 Qwen3-VL-32B-Instruct 作为执行者 [25]。技能进化模型 F 使用 GPT-5 实例化。2 [26]或 Gemini-3-flash [27]。所有执行者参数在技能进化期间保持固定，因此性能提升来自进化后的技能，而不是模型参数更新。

基线。我们在主要结果中比较了 EmbodiSkill 与两组基线。首先，闭源的直接代理直接使用 GPT-5.2 或 Gemini-3-flash 执行任务，而不使用进化的技能。其次，基于记忆的方法，包括 Mem 0 [17], G-Memory [14], 和 LangMem [19], 存储和检索轨迹级信息来指导任务执行。对于使用相同本地执行者的方法，执行者保持相同，以便性能差异反映外部技能或记忆机制，而不是执行者的变化。

4.2. 主要结果

桌子 1 和桌子 2 报告了在ALFWorld和EmbodiedBench上的主要结果。总体而言，EmbodiSkill通过从训练轨迹中进化外部技能，提高了任务成功率，表现出更强的性能

Table 2 | Task success rate (%) on EmbodiedBench. Avg. reports the average success rate across subcategories, while subcategory scores are rounded to integers for readability. Ours-GPT and Ours-Gemini denote EmbodiSkill using GPT-5.2 and Gemini-3-flash as the skill evolution model, respectively. Bold indicates the best Avg. result for each benchmark; underline indicates the second-best Avg. result.

Method	Avg.	EB-Habitat						Avg.	EB-Navigation				
		Base	Com.	Comp.	Vis.	Spa.	Long		Base	Com.	Comp.	Vis.	Long
Closed-source direct agents													
GPT-5.2	40.00	80	40	26	44	32	18	<u>57.33</u>	63	72	68	65	18
Gemini-3-flash	46.00	78	42	38	56	32	30	56.00	62	60	55	47	57
Executor: Qwen3-VL-8B-Instruct													
Memory-based baselines													
No memory	24.00	58	20	20	18	18	10	45.00	58	53	62	50	2
Mem0	20.33	60	16	8	16	14	8	46.33	58	57	62	55	0
G-Memory	25.66	58	14	12	34	30	6	37.33	62	43	43	38	0
LangMem	19.67	60	12	8	16	14	8	47.00	60	57	62	57	0
EmbodiSkill (Ours)													
Ours-GPT	45.33	76	46	36	52	38	24	50.33	68	65	62	57	0
Ours-Gemini	41.00	78	34	32	52	34	16	49.00	60	63	67	52	3
Executor: Qwen3-VL-32B-Instruct													
Memory-based baselines													
No memory	34.67	78	26	28	34	28	14	46.33	60	53	60	53	5
Mem0	38.33	84	24	26	42	34	20	52.00	62	60	65	67	7
G-Memory	45.00	92	24	36	50	32	34	50.33	52	53	58	65	23
LangMem	38.33	86	26	28	40	32	18	52.00	62	60	65	67	7
EmbodiSkill (Ours)													
Ours-GPT	<u>50.33</u>	92	46	50	54	34	26	**61.33**	65	68	73	67	33
Ours-Gemini	**52.33**	96	44	52	62	36	24	**61.33**	67	70	70	68	32

than direct model execution and trajectory-level memory retrieval.

Results on ALFWorld. On ALFWorld, EmbodiSkill achieves the best overall performance among all evaluated methods. With Qwen3.5-27B as the executor and GPT-5.2 as the skill evolution model, EmbodiSkill reaches 93.28% task success. This outperforms GPT-5.2 used as a direct agent by 31.58% and Gemini-3-flash used as a direct agent by 13.63%. Compared with memory-based methods under the same Qwen3.5-27B executor, EmbodiSkill exceeds the strongest memory baseline, G-Memory, by 25.01%. These results indicate that consolidating training trajectories into an evolving skill provides stronger guidance than directly executing with a frontier model or retrieving trajectory-level memory.

The subtask breakdown further shows that the improvement is not concentrated in a single task type. EmbodiSkill with Qwen3.5-27B and GPT-5.2 achieves the best or tied-best performance on five of the six ALFWorld subtask categories, including Put, Clean, Cool, Examine, and Puttwo. The gain is especially clear on Puttwo, where EmbodiSkill reaches 100.00%, compared with 52.94% for G-Memory under the same executor. This suggests that the evolved skill is particularly helpful for multi-step household tasks that require object search, state tracking, and correct action ordering.

表 2 | 任务成功率 (%) 在 EmbodiedBench 上。平均报告了跨子类别的平均成功率，而子类别分数四舍五入为整数以提高可读性。Ours-GPT 和 Ours- Gemini 分别表示使用 GPT-5.2 和 Gemini-3-flash 作为技能进化模型的 EmbodiSkill。粗体表示每个基准的最佳平均结果；下划线表示第二好的平均结果。

方法	Avg.	EB-Habitat						Avg.	EB-Navigation				
		Base	Com.	Comp.	Vis.	Spa.	Long		Base	Com.	Comp.	Vis.	Long
闭源直接代理													
GPT-5.2	40.00	80	40	26	44	32	18	<u>57.33</u>	63	72	68	65	18
Gemini-3-flash	46.00	78	42	38	56	32	30	56.00	62	60	55	47	57
执行者：Qwen3-VL-8B-Instruct													
基于记忆的基线													
无记忆	24.00	58	20	20	18	18	10	45.00	58	53	62	50	2
Mem0	20.33	60	16	8	16	14	8	46.33	58	57	62	55	0
G-Memory	25.66	58	14	12	34	30	6	37.33	62	43	43	38	0
LangMem	19.67	60	12	8	16	14	8	47.00	60	57	62	57	0
EmbodiSkill (Ours)													
Ours-GPT	45.33	76	46	36	52	38	24	50.33	68	65	62	57	0
Ours-Gemini	41.00	78	34	32	52	34	16	49.00	60	63	67	52	3
执行者: Qwen3-VL-32B-Instruct													
基于记忆的基线													
无记忆	34.67	78	26	28	34	28	14	46.33	60	53	60	53	5
Mem0	38.33	84	24	26	42	34	20	52.00	62	60	65	67	7
G-Memory	45.00	92	24	36	50	32	34	50.33	52	53	58	65	23
LangMem	38.33	86	26	28	40	32	18	52.00	62	60	65	67	7
EmbodiSkill (Ours)													
Ours-GPT	<u>50.33</u>	92	46	50	54	34	26	**61.33**	65	68	73	67	33
Ours-Gemini	52.33	96	44	52	62	36	24	**61.33**	67	70	70	68	32

比直接模型执行和轨迹级记忆检索表现更好。

在ALFWorld上的结果。 在ALFWorld上，EmbodiSkill在所有评估方法中实现了最佳的整体性能。使用Qwen3.5-27B作为执行者和GPT-5.2作为技能进化模型，EmbodiSkill达到了93.28%的任务成功率。这比直接作为代理使用的GPT-5.2高31.58%，比直接作为代理使用的Gemini-3-flash高13.63%。与在相同Qwen3.5-27B执行者下的基于记忆的方法相比，EmbodiSkill超过了最强的记忆基线G-Memory，高25.01%。这些结果表明，将训练轨迹整合到一个进化的技能中，比直接使用前沿模型执行或检索轨迹级记忆提供了更强的指导。

子任务分解进一步表明，改进并非集中在单一任务类型上。使用Qwen3.5-27B和GPT-5.2的EmbodiSkill在六个ALFWorld子任务类别中的五个上取得了最佳或并列最佳性能，包括放置、清洁、冷却、检查和放置两件物品。在放置两件物品上，EmbodiSkill达到了100.00%，而G-Memory在相同执行者下为52.94%。这表明进化后的技能特别有助于需要物体搜索、状态跟踪和正确动作排序的多步家庭任务。

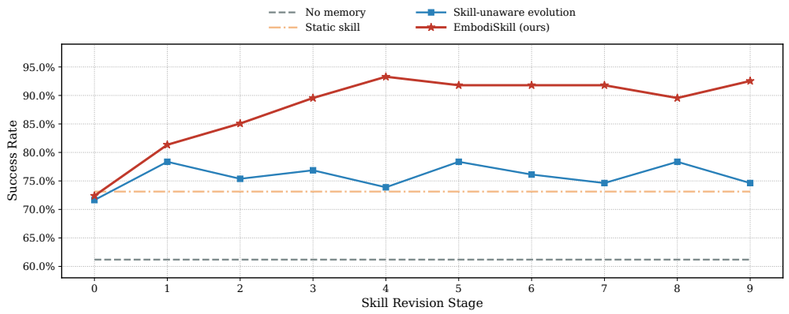


Figure 3 | ALFWorld test success rate across skill revision stages. EmbodiSkill quickly improves from the static skill and remains at a high success rate, while skill-unaware evolution converges to a lower performance range.

Results on EmbodiedBench. Table 2 reports results on visual embodied object interaction and navigation. On EB-Habitat, EmbodiSkill with Qwen3-VL-32B-Instruct and Gemini-3-flash achieves the best average score of 52.33%, outperforming the strongest memory-based baseline by 16.29% and the strongest closed-source direct agent by 13.76%. On EB-Navigation, EmbodiSkill with Qwen3-VL-32B-Instruct reaches the best average score of 61.33% with both GPT-5.2 and Gemini-3-flash as the skill evolution model, outperforming the strongest memory-based baseline by 17.94% and the strongest closed-source direct agent by 6.98%. These results show that EmbodiSkill also improves task success in visual embodied settings, where agents must rely on visual observation and spatial grounding to complete object interaction and navigation tasks.

Table 3 | Ablation of skill self-evolution and skill-aware reflection on ALFWorld. We report task success rate (%). Δ_{aware} denotes the improvement of EmbodiSkill over skill-unaware evolution.

Executor	Skill Model	Configurations				Δ_{aware}
		No skill	Static Skill	Skill-unaware	EmbodiSkill	
Qwen2.5-14B	GPT-5.2	46.27	65.67	70.90	86.57	+15.67
	Gemini-3-flash		58.95	67.91	85.82	**+17.91**
Qwen3.5-27B	GPT-5.2	61.19	73.13	78.36	**93.28**	+14.92
	Gemini-3-flash		79.85	85.82	87.31	+1.49

4.3. Ablation and Analysis

Ablation of skill self-evolution and skill-aware reflection. To analyze where the improvement of EmbodiSkill comes from, we compare four settings on ALFWorld. *No memory* directly uses the executor without external memory or skill evolution. *Static Skill* uses the initial skill without further revision. *Skill-unaware* updates the skill from trajectories, but revises the skill coarsely without explicitly identifying the implicated skill content, revision type, or separation between skill-body revision and skill-appendix update. EmbodiSkill performs the full skill-aware reflection and revision process.

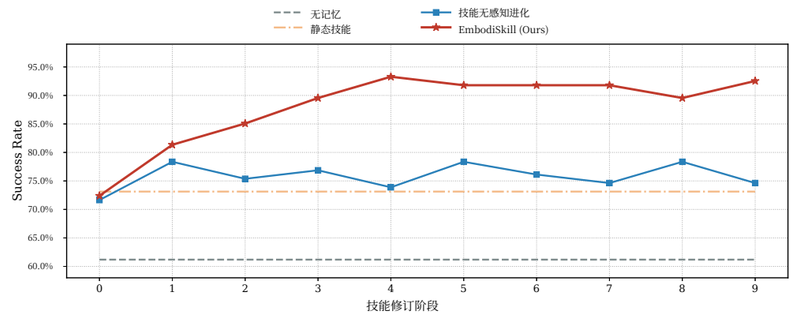


图 3 | ALFWorld测试成功率跨技能修订阶段。EmbodiSkill从静态技能快速提升，并保持高成功率，而技能无感知进化收敛到较低性能范围。

EmbodiedBench上的结果。 表 2 报告了视觉具身物体交互和导航的结果。在EB-Habitat上，使用Qwen3-VL-32B-Instruct和Gemini-3-flash的EmbodiSkill取得了52.33%的最佳平均分，比最强的基于记忆的基线高16.29%，比最强的闭源直接代理高13.76%。在EB-Navigation上，使用Qwen3-VL-32B-Instruct的EmbodiSkill在技能进化模型为GPT-5.2和Gemini-3-flash时达到了61.33%的最佳平均分，比最强的基于记忆的基线高17.94%，比最强的闭源直接代理高6.98%。这些结果表明，EmbodiSkill也提高了视觉具身环境中的任务成功率，在这些环境中，智能体必须依赖视觉观察和空间 grounding 来完成物体交互和导航任务。

表 3 | 技能自我进化和技能意识反射在ALFWorld上的消融。我们报告任务成功率（%）。 Δ_{aware} 表示EmbodiSkill相对于技能无感知进化的改进。

执行者	技能模型	配置				Δ_{aware}
		无技能	StaticSkill	技能-unaware	EmbodiSkill	
Qwen2.5-14B	GPT-5.2	46.27	65.67	70.90	86.57	+15.67
	Gemini-3-flash		58.95	67.91	85.82	**+17.91**
Qwen3.5-27B	GPT-5.2	61.19	73.13	78.36	**93.28**	+14.92
	Gemini-3-flash		79.85	85.82	87.31	+1.49

4.3. 消融和分析

技能自我进化和技能感知反思的消融。 为了分析 EmbodiSkill 改进的来源，我们在 ALFWorld 上比较了四种设置。 无记忆 直接使用执行者，不使用外部内存或技能进化。 静态技能 使用初始技能，不再进行修订。技能无感知 从轨迹中更新技能，但粗略地修订技能，而没有明确识别相关的技能内容、修订类型或技能体修订与技能附录更新之间的区别。EmbodiSkill 执行完整的技能感知反思和修订过程。

As shown in Table 3, both skill self-evolution and skill-aware reflection contribute to performance. With Qwen3.5-27B as the executor and GPT-5.2 as the skill evolution model, the static skill improves over the no-memory setting from 61.19% to 73.13%, corresponding to a 19.51% relative improvement and showing the benefit of skill-guided execution. Skill-unaware evolution further improves the result to 78.36%, but remains substantially below EmbodiSkill, which reaches 93.28%. This corresponds to a 27.55% relative improvement over the static skill and a 19.04% relative improvement over skill-unaware evolution. Similar gains are observed with Qwen2.5-14B-Instruct, where EmbodiSkill improves over skill-unaware evolution by 22.10% with GPT-5.2 and 26.37% with Gemini-3-flash. These results show that the gain does not come only from updating the skill, but from making the update skill-aware and targeted.

Evolution over skill revision stages. We further examine how performance changes during skill self-evolution. Figure 3 shows the ALFWorld test success rate across 10 skill revision stages under the Qwen3.5-27B executor and GPT-5.2 skill evolution model. EmbodiSkill rapidly improves from the static skill performance of 73.13% and reaches 93.28% during evolution, after which it remains in a high performance range. In contrast, skill-unaware evolution improves over the static skill but converges to a lower performance range and exhibits larger fluctuations. This suggests that directly revising the skill without skill-aware attribution can make evolution less stable, while EmbodiSkill produces more reliable improvement by updating targeted skill content.

5. Conclusion

We presented EmbodiSkill, a framework for embodied skill self-evolution. EmbodiSkill uses skill-aware reflection to examine trajectories with respect to the current skill and produce targeted revision signals, avoiding coarse whole-skill rewriting. These signals are integrated through a Skill-Aware Evolution Spiral, where revised skills guide subsequent task execution and new trajectories provide further evidence for skill improvement. Experiments on ALFWorld and EmbodiedBench show that EmbodiSkill improves task success over direct agents and memory-based baselines, and ablations confirm the importance of skill-aware revision. These results suggest that skill-aware self-evolution is a promising way to build embodied agents that accumulate reusable procedural knowledge from their own trajectories.

References

[1] Mohit Shridhar, Jesse Thomason, Daniel Gordon, Yonatan Bisk, Winson Han, Roozbeh Mottaghi, Luke Zettlemoyer, and Dieter Fox. ALFRED: A benchmark for interpreting grounded instructions for everyday tasks. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 10740–10749, June 2020.

[2] Mohit Shridhar, Xingdi Yuan, Marc-Alexandre Côté, Yonatan Bisk, Adam Trischler, and Matthew Hausknecht. ALFWorld: Aligning text and embodied environments for interactive learning. In *International Conference on Learning Representations*, 2021.

[3] Rui Yang, Hanyang Chen, Junyu Zhang, Mark Zhao, Cheng Qian, Kangrui Wang, Qineng Wang, Teja Venkat Koripella, Marziyeh Movahedi, Manling Li, Heng Ji, Huan Zhang, and Tong Zhang. EmbodiedBench: Comprehensive benchmarking multi-modal large language models for vision-driven embodied agents. In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, pages 70576–70631. PMLR, 2025.

如表 3所示，技能自我进化和技能感知反思都对性能有贡献。以 Qwen3.5-27B 作为执行者，GPT-5.2 作为技能进化模型，静态技能从 61.19% 提升至 73.13%，对应 19.51% 的相对提升，显示了技能引导执行的好处。技能无感知进化进一步将结果提升至 78.36%，但仍然远低于 EmbodiSkill，后者达到 93.28%。这对应于相对于静态技能 27.55% 的相对提升和相对于技能无感知进化 19.04% 的相对提升。使用 Qwen2.5-14B-Instruct 时观察到类似的收益，其中 EmbodiSkill 在使用 GPT-5.2 和 Gemini-3-flash 时分别比技能无感知进化提升了 22.10% 和 26.37%。这些结果表明，收益不仅来自更新技能，还来自使更新技能感知和有针对性。

在技能修订阶段进行进化。我们进一步检查了技能自我进化过程中性能的变化。图 3 展示了在Qwen3.5-27B执行器和GPT-5.2技能进化模型下，ALFWorld测试成功率在10个技能修订阶段的变化。EmbodiSkill从73.13%的静态技能性能迅速提升，并在进化过程中达到93.28%，之后保持在较高的性能范围内。相比之下，技能无感知进化在静态技能基础上有所提升，但收敛到较低的性能范围并表现出更大的波动。这表明直接修订技能而不进行技能感知归因会使进化过程不稳定，而EmbodiSkill通过更新目标技能内容产生了更可靠的改进。

5. 结论

我们提出了EmbodiSkill，一个用于具身技能自我进化的框架。EmbodiSkill使用技能感知反思来检查与当前技能相关的轨迹，并生成目标修正信号，避免粗粒度的全技能重写。这些信号通过技能感知进化螺旋进行整合，其中修订后的技能指导后续的任务执行，新的轨迹则为技能改进提供进一步证据。在ALFWorld和EmbodiedBench上的实验表明，EmbodiSkill比直接代理和基于记忆的基线提高了任务成功率，消融实验证实了技能感知修订的重要性。这些结果表明，技能感知自我进化是构建具身智能体的一种有前景的方法，这些智能体能够从自己的轨迹中积累可重用的程序性知识。

参考文献

[1] Mohit Shridhar, Jesse Thomason, Daniel Gordon, Yonatan Bisk, Winson Han, Roozbeh Mottaghi, Luke Zettlemoyer, and Dieter Fox. ALFRED: 一个用于解释日常任务中具身指令的基准。在 *IEEE/CVF*计算机视觉与模式识别会议论文集，第10740–10749页，2020年6月.[2] Mohit Shridhar, Xingdi Yuan, Marc-Alexandre Côté, Yonatan Bisk, Adam Trischler, 和 Matthew Hausknecht. ALFWorld: 对齐文本和具身环境以进行交互式学习。在 学习表示国际会议，2021年.[3] Rui Yang, Hanyang Chen, Junyu Zhang, Mark Zhao, Cheng Qian, Kangrui Wang, Qineng Wang, Teja Venkat Koripella, Marziyeh Movahedi, Manling Li, Heng Ji, Huan Zhang, 和 Tong Zhang. EmbodiedBench: 用于视觉驱动具身智能体的多模态大型语言模型的综合基准。在 第*42*届国际机器学习会议论文集，第267卷 机器学习研究论文集，第70576–70631页。PMLR, 2025.

[4] Brian Ichter, Anthony Brohan, Yevgen Chebotar, Chelsea Finn, Karol Hausman, Alexander Herzog, Daniel Ho, Julian Ibarz, Alex Irpan, Eric Jang, Ryan Julian, Dmitry Kalashnikov, Sergey Levine, Yao Lu, Carolina Parada, Kanishka Rao, Pierre Sermanet, Alexander T. Toshev, Vincent Vanhoucke, Fei Xia, Ted Xiao, Peng Xu, Mengyuan Yan, Noah Brown, Michael Ahn, Omar Cortes, Nicolas Sievers, Clayton Tan, Sichun Xu, Diego Reyes, Jarek Rettinghouse, Jornell Quiambao, Peter Pastor, Linda Luu, Kuang-Huei Lee, Yuheng Kuang, Sally Jesmonth, Nikhil J. Joshi, Kyle Jeffrey, Rosario Jauregui Ruano, Jasmine Hsu, Keerthana Gopalakrishnan, Byron David, Andy Zeng, and Chuyuan Kelly Fu. Do as i can, not as i say: Grounding language in robotic affordances. In *Proceedings of The 6th Conference on Robot Learning*, volume 205 of *Proceedings of Machine Learning Research*, pages 287–318. PMLR, 2023.

[5] Jacky Liang, Wenlong Huang, Fei Xia, Peng Xu, Karol Hausman, Brian Ichter, Pete Florence, and Andy Zeng. Code as policies: Language model programs for embodied control. In *2023 IEEE International Conference on Robotics and Automation*, pages 9493–9500. IEEE, 2023.

[6] Ishika Singh, Valts Blukis, Arsalan Mousavian, Ankit Goyal, Danfei Xu, Jonathan Tremblay, Dieter Fox, Jesse Thomason, and Animesh Garg. ProgPrompt: program generation for situated robot task planning using large language models. *Autonomous Robots*, 47:999–1012, 2023.

[7] Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. Voyager: An open-ended embodied agent with large language models. *Transactions on Machine Learning Research*, 2024.

[8] Runnan Fang, Yuan Liang, Xiaobin Wang, Jialong Wu, Shuofei Qiao, Pengjun Xie, Fei Huang, Huajun Chen, and Ningyu Zhang. Memp: Exploring agent procedural memory. In *Findings of the Association for Computational Linguistics: ACL 2026*, 2026. Accepted to ACL 2026 Findings. arXiv:2508.06433.

[9] Zouying Cao, Jiaji Deng, Li Yu, Weikang Zhou, Zhaoyang Liu, Bolin Ding, and Hai Zhao. Remember me, refine me: A dynamic procedural memory framework for experience-driven agent evolution. In *Findings of the Association for Computational Linguistics: ACL 2026*, 2026. Accepted to ACL 2026 Findings. arXiv:2512.10696.

[10] Jingwei Ni, Yihao Liu, Xinpeng Liu, Yutao Sun, Mengyu Zhou, Pengyu Cheng, Dexin Wang, Erchao Zhao, Xiaoxi Jiang, and Guanjun Jiang. Trace2Skill: Distill trajectory-local lessons into transferable agent skills, 2026.

[11] Hanrong Zhang, Shicheng Fan, Henry Peng Zou, Yankai Chen, Zhenting Wang, Jiayu Zhou, Chengze Li, Wei-Chieh Huang, Yifei Yao, Kening Zheng, Xue Liu, Xiaoxiao Li, and Philip S. Yu. CoEvoSkills: Self-evolving agent skills via co-evolutionary verification, 2026.

[12] Qirui Mi, Zhijian Ma, Mengyue Yang, Haoxuan Li, Yisen Wang, Haifeng Zhang, and Jun Wang. Skill-Pro: Learning reusable skills from experience via non-parametric PPO for LLM agents, 2026.

[13] Wenlong Huang, Pieter Abbeel, Deepak Pathak, and Igor Mordatch. Language models as zero-shot planners: Extracting actionable knowledge for embodied agents. In *Proceedings of the 39th International Conference on Machine Learning*, volume 162 of *Proceedings of Machine Learning Research*, pages 9118–9147. PMLR, 2022.

[14] Guibin Zhang, Muxin Fu, Kun Wang, Frank Wan, Miao Yu, and Shuicheng Yan. G-Memory: Tracing hierarchical memory for multi-agent systems. In *Advances in Neural Information Processing Systems*, volume 38, pages 12988–13018. Curran Associates, Inc., 2025.

[15] Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems*, volume 36, pages 8634–8652. Curran Associates, Inc., 2023.

[16] Andrew Zhao, Daniel Huang, Quentin Xu, Matthieu Lin, Yong-Jin Liu, and Gao Huang. ExpeL: LLM agents are experiential learners. *Proceedings of the AAAI Conference on Artificial Intelligence*, 38(17):19632–19642, 2024.

[17] Prateek Chhikara, Dev Khant, Saket Aryan, Taranjeet Singh, and Deshraj Yadav. Mem0: Building production-ready AI agents with scalable long-term memory, 2025.

[4] Brian Ichter, Anthony Brohan, Yevgen Chebotar, Chelsea Finn, Karol Hausman, Alexander Herzog, Daniel Ho, Julian Ibarz, Alex Irpan, Eric Jang, Ryan Julian, Dmitry Kalashnikov, Sergey Levine, Yao Lu, Carolina Parada, Kanishka Rao, Pierre Sermanet, Alexander T. Toshev, Vincent Vanhoucke, Fei Xia, Ted Xiao, Peng Xu, Mengyuan Yan, Noah Brown, Michael Ahn, Omar Cortes, Nicolas Sievers, Clayton Tan, Sichun Xu, Diego Reyes, Jarek Rettinghouse, Jornell Quiambao, Peter Pastor, Linda Luu, Kuang-Huei Lee, Yuheng Kuang, Sally Jesmonth, Nikhil J. Joshi, Kyle Jeffrey, Rosario Jauregui Ruano, Jasmine Hsu, Keerthana Gopalakrishnan, Byron David, Andy Zeng, 和 Chuyuan Kelly Fu. 做我所能，而非我所言：将语言与机器人可供性相结合。在 机器人学习第6届会议论文集, 第205卷 机器学习研究论文集，第287–318页。PMLR, 2023.[5] Jacky Liang, Wenlong Huang, Fei Xia, Peng Xu, Karol Hausman, Brian Ichter, Pete Florence, 和 Andy Zeng. 代码作为策略：用于具身控制的语言模型程序。在 2023年IEEE国际机器人与自动化会议，第9493–9500页。IEEE, 2023.[6] Ishika Singh, Valts Blukis, Arsalan Mousavian, Ankit Goyal, Danfei Xu, Jonathan Tremblay, Dieter Fox, Jesse Thomason, 和 Animesh Garg. ProgPrompt：使用大型语言模型进行具身机器人任务规划的程序生成。自主机器人, 47:999–1012, 2023.[7] Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, 和 Anima Anandkumar. Voyager：一个使用大型语言模型的开放式具身智能体。机器学习研究交易, 2024.[8] Runnan Fang, Yuan Liang, Xiaobin Wang, Jialong Wu, Shuofei Qiao, Pengjun Xie, Fei Huang, Huajun Chen, 和 Ningyu Zhang. Memp：探索智能体程序记忆。在 计算语言学协会发现：ACL 2026，2026年。已接受ACL 2026发现。arXiv:2508.06433.[9] Zouying Cao, Jiaji Deng, Li Yu, Weikang Zhou, Zhaoyang Liu, Bolin Ding, 和 Hai Zhao. 记住我，完善我：一个用于经验驱动智能体进化的动态程序记忆框架。在 计算语言学协会发现：ACL 2026，2026年。已接受ACL 2026发现。arXiv:2512.10696.[10] Jingwei Ni, Yihao Liu, Xinpeng Liu, Yutao Sun, Mengyu Zhou, Pengyu Cheng, Dexin Wang, Erchao Zhao, Xiaoxi Jiang, 和 Guanjun Jiang. Trace2Skill：将轨迹局部经验蒸馏为可转移的代理技能，2026年.[11] Hanrong Zhang, Shicheng Fan, Henry Peng Zou, Yankai Chen, Zhenting Wang, Jiayu Zhou, Chengze Li, Wei-Chieh Huang, Yifei Yao, Kening Zheng, Xue Liu, Xiaoxiao Li, 和 Philip S. Yu. CoEvoSkills：通过协同进化验证进行自我进化的代理技能，2026年.[12] Qirui Mi, Zhijian Ma, Mengyue Yang, Haoxuan Li, Yisen Wang, Haifeng Zhang, 和 Jun Wang. Skill-Pro：通过非参数PPO从经验中学习可重用技能，用于LLM智能体，2026年.[13] Wenlong Huang, Pieter Abbeel, Deepak Pathak, 和 Igor Mordatch. 语言模型作为零样本规划器：为具身智能体提取可执行知识。在 第39届国际机器学习会议论文集，第162卷 机器学习研究论文集，第9118–9147页。PMLR, 2022.
[14] Guibin Zhang, Muxin Fu, Kun Wang, Frank Wan, Miao Yu, 和 Shuicheng Yan. G-Memory：为多智能体系统追踪分层记忆。在 神经信息处理系统进展，第38卷，第12988–13018页。Curran Associates, Inc., 2025.[15] Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, 和 Shunyu Yao. Reflexion：具有语言强化学习的语言智能体。在 神经信息处理系统进展，第36卷，第8634–8652页。Curran Associates, Inc., 2023.[16] Andrew Zhao, Daniel Huang, Quentin Xu, Matthieu Lin, Yong-Jin Liu, 和 Gao Huang. ExpeL：LLM智能体是经验学习者。人工智能AAAI会议论文集, 38(17):19632–19642, 2024.[17] Prateek Chhikara, Dev Khant, Saket Aryan, Taranjeet Singh, 和 Deshraj Yadav. Mem0：构建生产就绪的AI智能体，具有可扩展的长期记忆，2025年。

[18] Wujiang Xu, Zujie Liang, Kai Mei, Hang Gao, Juntao Tan, and Yongfeng Zhang. A-Mem: Agentic memory for LLM agents. In *Advances in Neural Information Processing Systems*, volume 38, pages 17577–17604. Curran Associates, Inc., 2025.

[19] LangChain AI. Langmem: A framework for long-term memory in llm agents. https://github.com/langchain-ai/langmem, 2026. Accessed: 2026-05-04.

[20] Peng Xia, Jianwen Chen, Hanyang Wang, Jiaqi Liu, Kaide Zeng, Yu Wang, Siwei Han, Yiyang Zhou, Xujiang Zhao, Haifeng Chen, Zeyu Zheng, Cihang Xie, and Huaxiu Yao. SkillRL: Evolving agents via recursive skill-augmented reinforcement learning, 2026.

[21] Haozhen Zhang, Quanyu Long, Jianzhu Bao, Tao Feng, Weizhi Zhang, Haodong Yue, and Wenya Wang. MemSkill: Learning and evolving memory skills for self-evolving agents, 2026.

[22] Guanyu Jiang, Zhaochen Su, Xiaoye Qu, and Yi R. Fung. XSkill: Continual learning from experience and skills in multimodal agents. In *International Conference on Machine Learning*, 2026. Accepted to ICML 2026. arXiv:2603.12056.

[23] An Yang, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chengyuan Li, Dayiheng Liu, Fei Huang, Haoran Wei, Huan Lin, Jian Yang, Jianhong Tu, Jianwei Zhang, Jianxin Yang, Jiaxi Yang, Jingren Zhou, Junyang Lin, Kai Dang, Keming Lu, Keqin Bao, Kexin Yang, Le Yu, Mei Li, Mingfeng Xue, Pei Zhang, Qin Zhu, Rui Men, Runji Lin, Tianhao Li, Tianyi Tang, Tingyu Xia, Xingzhang Ren, Xuancheng Ren, Yang Fan, Yang Su, Yichang Zhang, Yu Wan, Yuqiong Liu, Zeyu Cui, Zhenru Zhang, and Zihan Qiu. Qwen2.5 technical report, 2024.

[24] Qwen Team. Qwen3.5: Towards native multimodal agents, February 2026.

[25] Shuai Bai, Yuxuan Cai, Ruizhe Chen, Keqin Chen, Xionghui Chen, Zesen Cheng, Lianghao Deng, Wei Ding, Chang Gao, Chunjiang Ge, Wenbin Ge, Zhifang Guo, Qidong Huang, Jie Huang, Fei Huang, Binyuan Hui, Shutong Jiang, Zhaohai Li, Mingsheng Li, Mei Li, Kaixin Li, Zicheng Lin, Junyang Lin, Xuejing Liu, Jiawei Liu, Chenglong Liu, Yang Liu, Dayiheng Liu, Shixuan Liu, Dunjie Lu, Ruilin Luo, Chenxu Lv, Rui Men, Lingchen Meng, Xuancheng Ren, Xingzhang Ren, Sibo Song, Yuchong Sun, Jun Tang, Jianhong Tu, Jianqiang Wan, Peng Wang, Pengfei Wang, Qiuyue Wang, Yuxuan Wang, Tianbao Xie, Yiheng Xu, Haiyang Xu, Jin Xu, Zhibo Yang, Mingkun Yang, Jianxin Yang, An Yang, Bowen Yu, Fei Zhang, Hang Zhang, Xi Zhang, Bo Zheng, Humen Zhong, Jingren Zhou, Fan Zhou, Jing Zhou, Yuanzhi Zhu, and Ke Zhu. Qwen3-VL technical report, 2025.

[26] OpenAI. Update to GPT-5 system card: GPT-5.2. OpenAI system card update, December 2025.

[27] Google DeepMind. Gemini 3 Flash model card. Model card, December 2025.

[18] 徐吴江, 梁祖杰, 梅凯, 高航, 谭俊涛和张永峰. A-MEM：为LLM智能体的代理记忆. 在 神经信息处理系统进展， 第38卷， 第17577-17604页. Curran Associates, Inc., 2025.[19] LangChain AI. LangMem：为LLM智能体长期记忆的框架。 https://github.com/langchain-ai/langmem， 2026. 访问日期：2026-05-04.
[20] 夏鹏, 陈建文, 王汉阳, 刘佳琪, 曾凯德, 王宇, 韩思伟, 周一阳, 赵徐江, 陈海峰, 郑泽宇, 谢持航和姚华秀. SkillRL：通过递归技能增强强化学习进化智能体, 2026.[21] 张浩臻, 龙全宇, 鲍建珠, 冯涛, 张伟志, 岳浩东和王子涵. MemSkill：为自进化智能体学习和进化记忆技能, 2026.[22] 蒋冠宇, 苏赵辰, 曲晓叶和冯毅R. XSkill：多模态智能体从经验和技能中持续学习. 在 机器学习国际会议， 2026. 已接受ICML 2026. arXiv:2603.12056.
[23] 杨安, 杨宝松, 张贝辰, 会宾元, 郑博, 余 Bowen, 李成元, 刘代恒, 黄飞, 魏浩然, 林欢, 杨建, 涂建红, 张建伟, 陈建文, 杨建新, 杨嘉熙, 周景仁, 林俊阳, 党凯, 陆克明, 鲍克勤, 陈克勤, 杨克欣, 余乐, 李梅, 薛明峰, 杨明坤, 李明升, OpenAI, 张佩, 王鹏, 夏鹏, 王鹏飞, 黄奇栋, 朱秦, 王秋月, 龙全宇, Qwen团队, Qwen2.5, Qwen3.5, 白帅, 蔡宇轩 , 陈瑞哲 , 林润吉 , 刘诗璇 , 白帅 , 蒋舒通 , 宋思博 , 韩思伟 , 杨帆 , 刘阳 , 苏阳 , 张一昌 , 徐一恒 , 万宇 , 王宇, 朱元智, 孙宇冲, 刘宇琼, 蔡宇轩, 王宇轩, 程泽森, 崔泽宇, 郑泽宇, 苏赵辰, 李兆海, 张振儒, 杨志博, 郭志方, 林志成 , 邱志涵 , 梁祖杰 , 杨安 , 余 Bowen, 张飞 , 张航 , 王汉阳 , 葛春江 , 葛文斌 , 郭志方 , 黄奇栋 , 黄杰 , 黄飞, 会宾元, 蒋舒通, 李兆海, 李明升, 李梅, 李凯欣, 林志成, 林俊阳, 刘雪静, 刘佳伟, 刘成龙, 刘阳, 刘代恒, 刘诗璇, 刘宇琼, 陆东杰, 罗瑞林, 吕晨旭, 门瑞, 任兴章, 任宣成, 宋思博, 孙宇冲, 唐俊, 涂建红, 万建强, 王鹏, 王鹏飞, 王秋月, 王宇, 王宇轩, 谢天宝, 徐海阳, 徐吴江, 徐金, 徐一恒, 杨帆, 杨建, 杨建新, 杨嘉熙, 杨明坤, 杨克欣, 杨安, 余 Bowen, 余乐, 岳浩东, 蔡宇轩, 曾凯德, 曾凯德, 薛明峰, 冯涛, 冯毅R., 魏浩然, 谢持航, 姚华秀, 郑博, 钟惠民, 周景仁, 周帆, 周一阳, 周静, 朱元智, 朱科. Qwen3-VL技术报告, 2025.[26] OpenAI. GPT-5系统卡更新：GPT-5.2. OpenAI系统卡更新, 2025年12月.[27] Google DeepMind. Gemini 3 Flash模型卡. 模型卡, 2025年12月.